15. Functions, Arguments, and Exit Codes in Scripts

15. Functions, Arguments, and Exit Codes in Scripts

Phase: Phase 5 — Shell scripting
Estimated time: 1–2 days
Slug: `15-bash-functions-args`

Learning objectives

- Define functions and local variables
- Validate arguments
- Return meaningful exit codes
- Source libraries carefully

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

Once a script grows past ten lines you want named, reusable pieces. Bash functions give you that, but they behave differently from functions in most languages: they do not return values, they return an exit status between 0 and 255. Data comes back through stdout (captured with `$(...)`) or through global variables. Callers decide what to do by checking `$?` or using the function directly in an `if`.

Exit codes are the shared language of Unix tools. `grep` returns 1 when nothing matches, `ssh` returns 255 on connection failure, `curl` returns 22 on HTTP errors with `-f`. Your functions should follow the same convention so that other scripts, cron, and CI can react correctly.

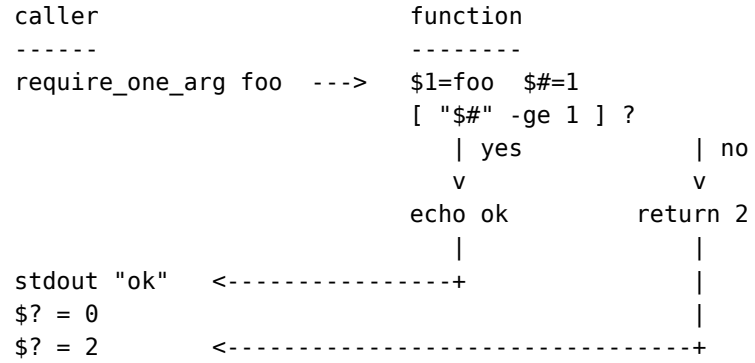
Core concepts

Concept	Syntax	Meaning
Define function	`name() { ...; }`	Body runs in the current shell
Arguments	`$1 $2 $#`	Inside a function these are the function's args
Return status	`return 2`	Sets `$?` for the caller (0-255)
Output value	`echo ok`	Caller captures with `$(fn)`
Local variable	`local x=1`	Scope limited to the function
Arg count check	`[ "$#" -ge 1 ]`	Test numerically
Usage error code	`2`	Convention: misuse of command
`exit` vs `return`	`exit 2`	`exit` ends the whole script

Common exit codes:

Code	Convention
0	Success
1	General failure
2	Misuse, bad arguments
126	Found but not executable
127	Command not found
130	Killed by Ctrl+C

Data flow of a call:



Worked examples

A function that checks its arguments:

```
require_one_arg() {
  if [ "$#" -ge 1 ]; then
    echo ok
    return 0
  fi
  echo "usage: require_one_arg VALUE" >&2
  return 2
}

require_one_arg hello
echo "status=$?"
require_one_arg
echo "status=$?"
```

Sample output:

```
ok
status=0
usage: require_one_arg VALUE
status=2
```

Using the status in an `if`, and capturing stdout:

```
if require_one_arg x >/dev/null; then echo accepted; fi
result=$(require_one_arg x)
echo "captured=$result"
```

Sample output:

```
accepted
captured=ok
```

Local versus global variables:

```
counter=1
bump() { local counter=99; counter=$((counter+1)); echo "in: $counter"; }
bump
echo "out: $counter"
```

Sample output:

```
in: 100
out: 1
```

Forwarding all arguments safely:

```
wrapper() { printf '[%s]\n' "$@"; }
wrapper "a b" c
```

Sample output:

```
[a b]
[c]
```

Mini-lab

1. Create `lib.sh` and put `require_one_arg` in it.
2. Source it: `source ./lib.sh`.
3. Call it with zero, one, and three arguments and print `$?` each time.
4. Send the usage message to stderr and prove it with `require_one_arg 2>/dev/null` (message disappears) and `require_one_arg 2>&1` (message shows).
5. Write `greet_or_fail` that calls `require_one_arg "$1"` and exits the script with the same status if it fails.
6. Remove `local` from a variable inside a function and observe the leak into the caller.
7. Test with `[ "$#" -ge 1 ]` versus `[ -n "$1" ]` and pass an empty string `""` to see the difference.
8. Run `echo $?` twice in a row and explain why the second result is 0.

Decision guide / failure modes

Symptom	Likely cause	Fix
`$?` is always 0	You read it after another command	Capture `rc=$?` immediately
Whole script exits on error	Used `exit` in a function	Use `return` in functions
Function prints but caller gets nothing	Value was echoed, not captured	`x=$(fn)`
Variable changed unexpectedly	Missing `local`	Declare locals
`[: too many arguments`	Unquoted `$1` with spaces	Quote: `[ -n "$1" ]`
Exit code 256 wraps to 0	Codes are modulo 256	Stay within 0-255

Return versus echo: use `return` for success or failure, `echo` for data. Never mix them in a way that makes the output ambiguous; send diagnostics to stderr with `>&2` so stdout stays clean for capture.

Rule of thumb: validate arguments first, fail fast with code 2, document the usage line, and keep functions small enough to test on their own.

Common mistakes

- Using exit inside sourced files unexpectedly
- Global variables everywhere
- Swallowing errors with `|| true` habitually

Exercises

Work in `exercises/15-bash-functions-args/`. Read that folder's `README.md` fully before coding.

```
cd exercises/15-bash-functions-args
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement require_one_arg in exercise.sh. If the function receives at least one argument it must print the word ok and return 0. If it receives no arguments it must return 2 and should not print ok. You may print a usage message to stderr (use >&2) but nothing on stdout in the failure case. Count arguments with `$#` and compare numerically with `[ "$#" -ge 1 ]`. Use return, not exit, so the sourcing shell is not killed. After ./test.sh passes, call the function by hand and read $? right after each call.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/15-bash-functions-args/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. How require at least one arg?
2. local keyword purpose?
3. When source vs execute?

Further reading

- https://mywiki.wooledge.org/BashFAQ